

Unit 1: Introduction to Database Management system and ER Model

Data - Fact that can be recorded or stored

->e.g. Person Name, Age, Gender and Weight etc.

Information: When data is processed, organized, structured or presented in a given context so as to make it useful, it is called information.

Database: - Collection of **logically related data**

->e.g. Books Database in Library, Student Database in University etc.

Management - Manipulation, Searching and Security of data

->e.g. Viewing result in GTU website, Searching exam papers in GTU website etc.

System - Programs or tools used to manage database

-e.g. SQL Server Studio Express, Oracle etc.

DBMS - A Database Management System is a software for creating and managing databases.

Database Management System (DBMS) is a **software designed to define, manipulate, retrieve and manage data in a database.**

e.g. MS SQL Server, Oracle etc.

Metadata: Metadata is data about data. Data such as table name, column name, data type, authorized user and user access privileges for any table is called metadata for that table.

Data dictionary: Data dictionary is an information repository which contains metadata. It is usually a part of the system catalog.

Data warehouse: Data warehouse is an information repository which stored data. It is design to facilitate reporting and analysis.

Field:

- A field is a character or group of characters that have a specific meaning.
- It is also called a data item. It is represented in the database by a value.
- For Example, customer id, name, society and city are all fields for customer Data.

Record:

- A record is a collection of logically related fields.
- For examples, collection of fields (id, name, address & city) forms a record for customer.

- **Disadvantages of file system (file processing systems) compare to Database management system.**

1] Data Redundancy :

- It is possible that the same information may be duplicated in different files. This leads to data redundancy.
- Data redundancy results in memory wastage.
- For example, consider that some customers have both kinds of accounts - saving and current. In this case, data about customers such as name, address, e-mail and contact number will be duplicated in both files, saving accounts file and current account file.
- In other words, same information will be stored in two different locations (files). And, it wastes memory.

2] Data Inconsistency

- Due to data redundancy, it is possible that data may not be in consistent state.
- For example, consider that an address of some customer changes.
- And, that customer has both kinds of accounts.
- Now, it is possible that this changed address is updated in only one file, leaving address in another file as it is.
- As a result of this, same customer will have two different addresses in two different files, making data inconsistent.

3]Difficulty in Accessing Data

- Accessing data is not convenient and efficient in file processing system.
- For example, suppose, there is a program to find information about all customers. But , what if there is a need to find out all customers from some particular city. In this case, there are two choices here: One, find out all customers using available program, and then extract the needed customers manually. Second, develop new program to get required information. Both options are not satisfactory.
- For each and every different kind of data access, separate programs are required. This is neither convenient nor efficient.

4]Limited Data Sharing

- Data are scattered in various files.
- Different files may have different formats. And these files may be stored in different folders (directories) may be of different computers of different departments.
- So, due to this data isolation, it is difficult to share data among different applications

5]Integrity Problems

- Data integrity means that the data contained in the database is both correct and consistent. For this purpose, the data stored in database must satisfy certain types of constraints (rules).
- For example, a balance for any account must not be less than zero. Such constraints are enforced in the system by adding appropriate code in application programs. But, when new constraints are added, such as balance should not be less than Rs. 5000, application programs need to be changed. But, it is not an easy task to change programs whenever required.

6]Atomicity Problems

- Any operation on database must be atomic.
- This means, operation completes either 100% or 0%.
- For example, a fund transfer from one account to another must happen in its entirety. But, computer systems are vulnerable to failure, such as system crash, virus attack. If a system failure occurs during the execution of fund transfer operation, it may possible that amount to be transferred, say, Rs. 500, is debited from one account, but is not credited to another account.
- This leaves database in consistent state. But, it is difficult to ensure atomicity in a file processing system.

7]Concurrent Access Anomalies

- Multiple users are allowed to access data simultaneously (concurrently). This is for the sake of better performance and faster response.
- Consider an operation to debit (withdrawal) an account. The program reads the old balance, calculates the new balance, and writes new balance back to database. Suppose an account has a balance of Rs. 5000. Now, a concurrent withdrawal of Rs. 1000 and Rs. 2000 may leave the balance Rs. 4000 or Rs. 3000 depending upon their completion time rather than the correct value of Rs. 2000.
- Here, concurrent data access should be allowed under some supervision.
- But, due to lack of co-ordination among different application programs, this is not possible in file processing systems.

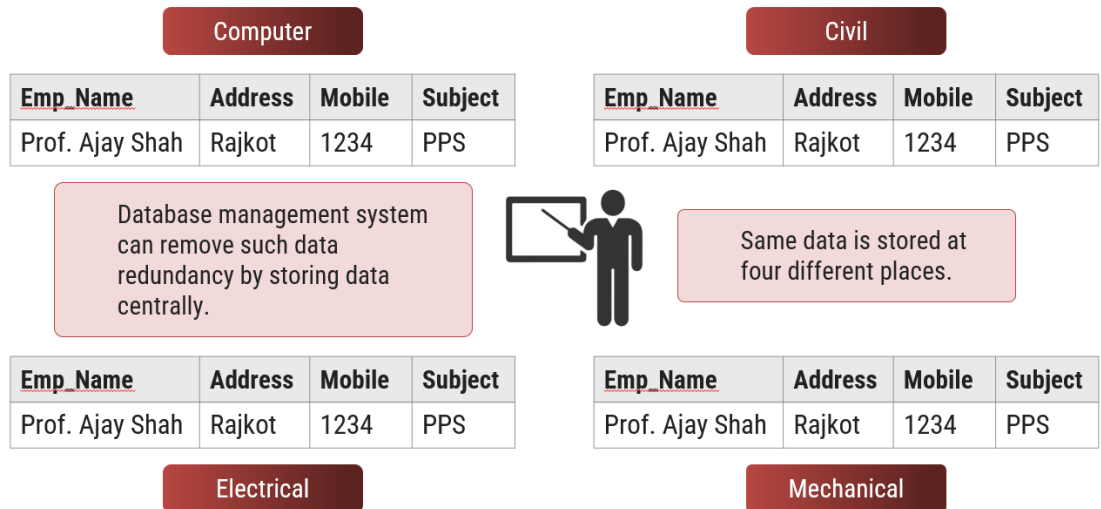
8]Security Problems

- Database should be accessible to users in a limited way.
- Each user should be allowed to access data concerning his application only.
- For example, a customer can check balance only for his/her own account. He/She should not have access for information about other accounts.
- But, in file processing system, application programs are added in an ad hoc manner by different programmers. So, it is difficult to enforce such kind of security constraints.

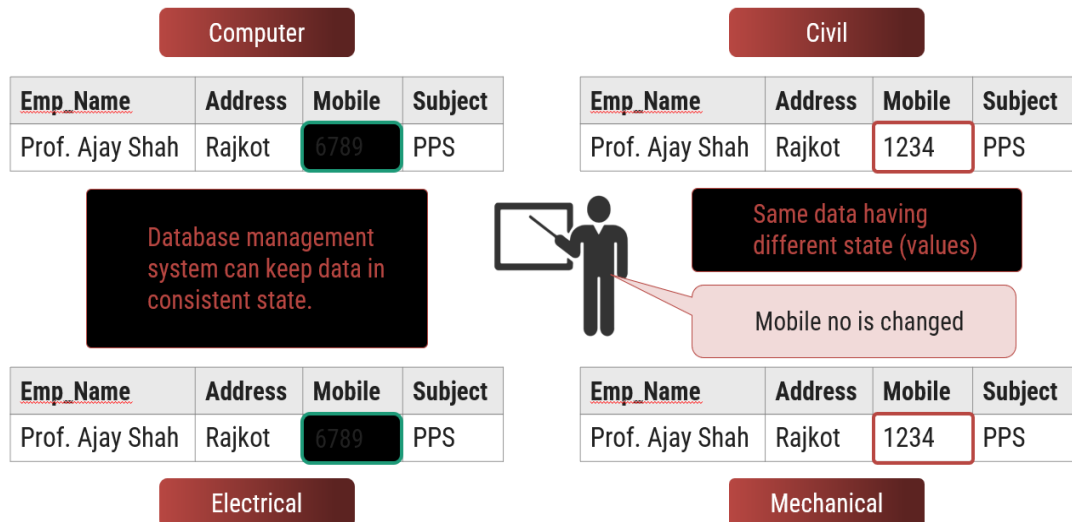
Advantages of DBMS

-

Reduce data redundancy (duplication)



Remove data inconsistency



Data isolation

File - 1

<u>Emp_Name</u>	Address	Mobile	Subject
Prof. Ajay Shah	Rajkot	1234	PPS

File - 2

<u>Emp_Name</u>	Post	Salary	Load
Prof. Ajay Shah	Lecturer	50,000	15

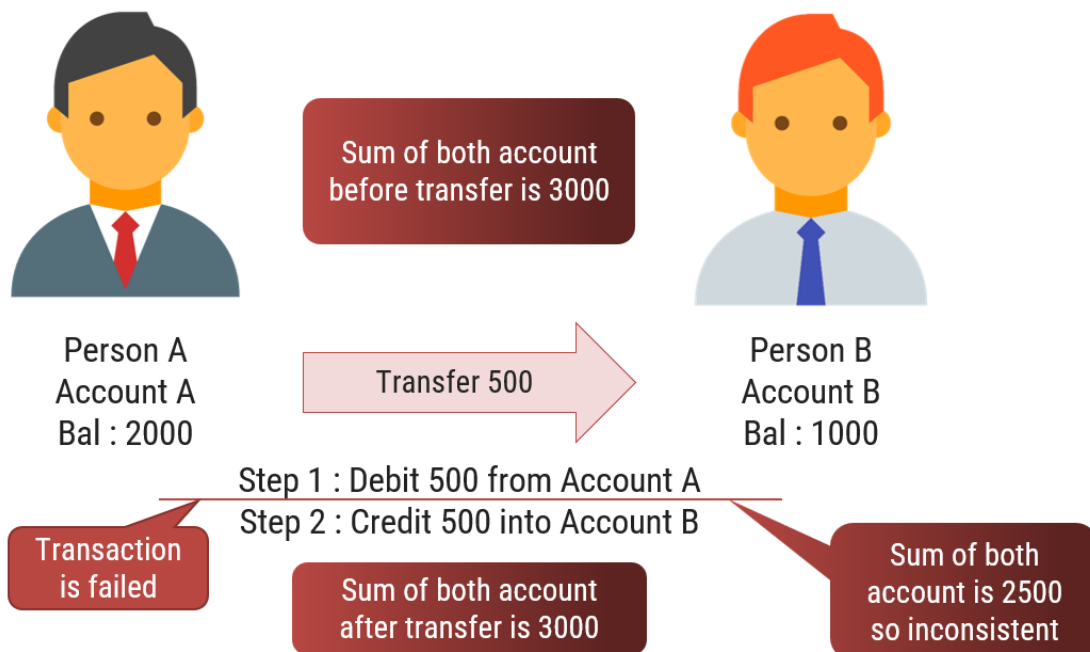
File - 3

<u>Emp_Name</u>	Teaching	Knowledge	Rating
Prof. Ajay Shah	Good	Excellent	9

- Data are scattered in various files.
- Files may be in different formats.
- Difficult to retrieve the appropriate data.

Guaranteed atomicity

Atomicity: Either transaction execute 0% or 100%.



Allow to implement integrity constraints

DBMS allows us to implement such business rules in our database..

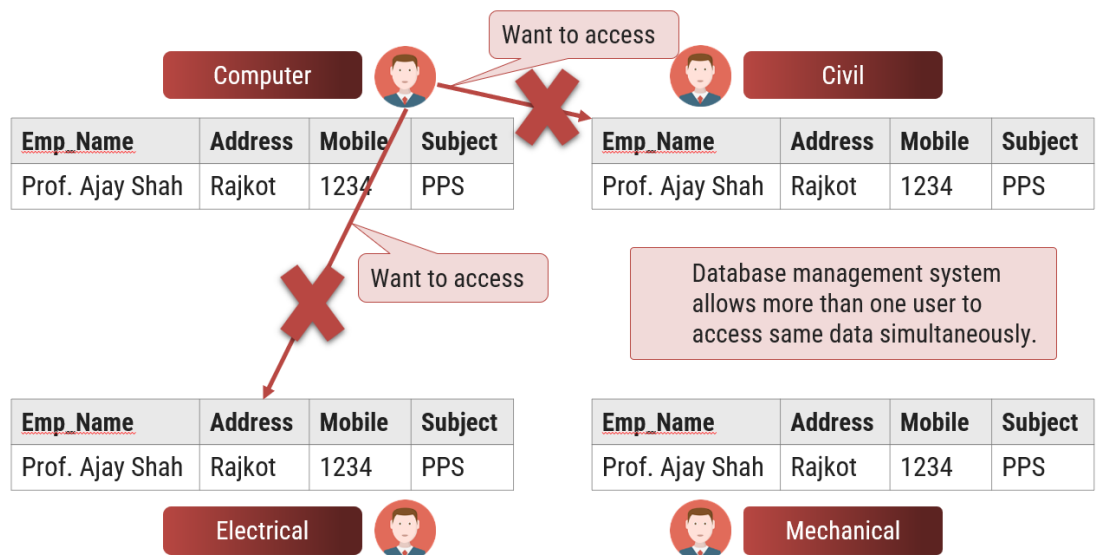
<u>Emp_Name</u>	Address	<u>Mobile_No</u>	Subject
Prof. Ajay Shah	Rajkot	9876543210	PPS

Should contain exact 10 digits

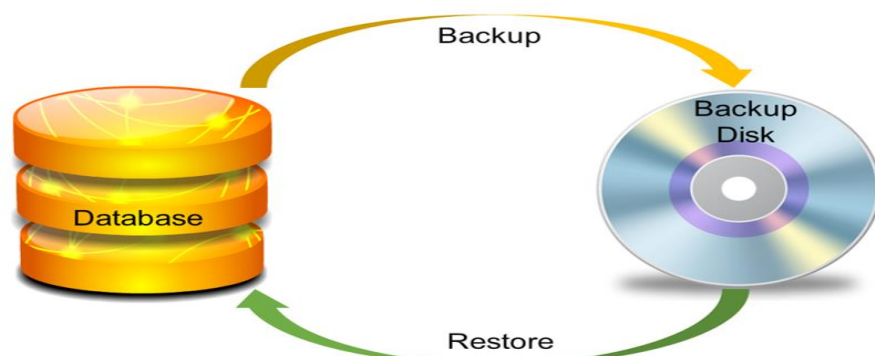
<u>Student_Name</u>	Branch	Backlog	SPI
<u>Nirav Patel</u>	Rajkot	0	8.5

Should be between 0 to 10

Sharing of data among multiple users



Providing backup and recovery services



Provides facilities to backup and restore the database in case of failure.

Reduce data redundancy (duplication)

- Avoids unnecessary duplication of data by storing data centrally.

Remove data inconsistency

- ↪ By eliminating redundancy, data inconsistency can be removed.

Data isolation

- ↪ A user can easily retrieve proper data as per his/her requirement.

Guaranteed atomicity

- ↪ Either transaction executes 0% or 100%.

Allow implementing integrity constraints

- ↪ Business rules can be implemented such as do not allow to store amount less than Rs. 0 in balance.

Sharing of data among multiple users

- ↪ More than one users can access same data at the same time.

Providing backup and recovery services

- ↪ Can take a regular auto or manual backup and use it to restore the database if it corrupts

▶ **Characteristics of Database approach**

The database approach has the following key characteristics:

1. Self-Describing Nature

- ▶ A database contains not only the data but also a description of the data structure (metadata).
- ▶ Metadata is stored in a data dictionary or catalog.

2. Program–Data Independence

- ▶ Data is separated from application programs.
- ▶ Changes in data structure do not require major changes in application programs.

3. Data Abstraction

- ▶ Users interact with the database without needing to know how data is physically stored.
- ▶ Provides different levels of views (physical, logical, and view levels).

4. Support for Multiple Views

- ▶ Different users can have different views of the same database according to their needs.
- ▶ Improves security and usability.

5. Data Sharing and Multiuser Transaction Processing

- ▶ Multiple users can access and manipulate data simultaneously.
- ▶ Concurrency control ensures data consistency and integrity.

6. Reduced Data Redundancy

- ▶ Data is stored centrally, minimizing duplication.
- ▶ Helps maintain consistency.

7. Data Integrity and Security

- ▶ Rules and constraints ensure accurate and valid data.
- ▶ Access controls protect data from unauthorized users.

8. Backup and Recovery

- ▶ Mechanisms are provided to recover data in case of system failures or data loss.
- ▶ In Short, The database approach emphasizes data independence, data sharing, reduced redundancy, security, integrity, multiple views, and efficient management of large amounts of data.

Applications of DBMS

- ▶ **DBMS is a computerized record-keeping system.**
- ▶ **DBMS is required where ever data need to be stored.**
 - ↳ E-Commerce (Flikart, Amazon, Shopclues, eBay etc...)
 - ↳ Online Television Streaming (Hotstar, Amazon Prime etc...)
 - ↳ Social Media (WhatsApp, Facebook, Twitter, LinkedIn etc...)
 - ↳ Banking & Insurance
 - ↳ Airline & Railway
 - ↳ Universities and Colleges/Schools
 - ↳ Library Management System
 - ↳ Human Resource Department
 - ↳ Hospitals and Medical Stores
 - ↳ Government Organizations
- ▶ **Metadata**
 - ↳ **Metadata is data about data.**
 - ↳ **Data such as table name, column name, data type, authorized user and user access privileges for any table is called metadata for that table.**

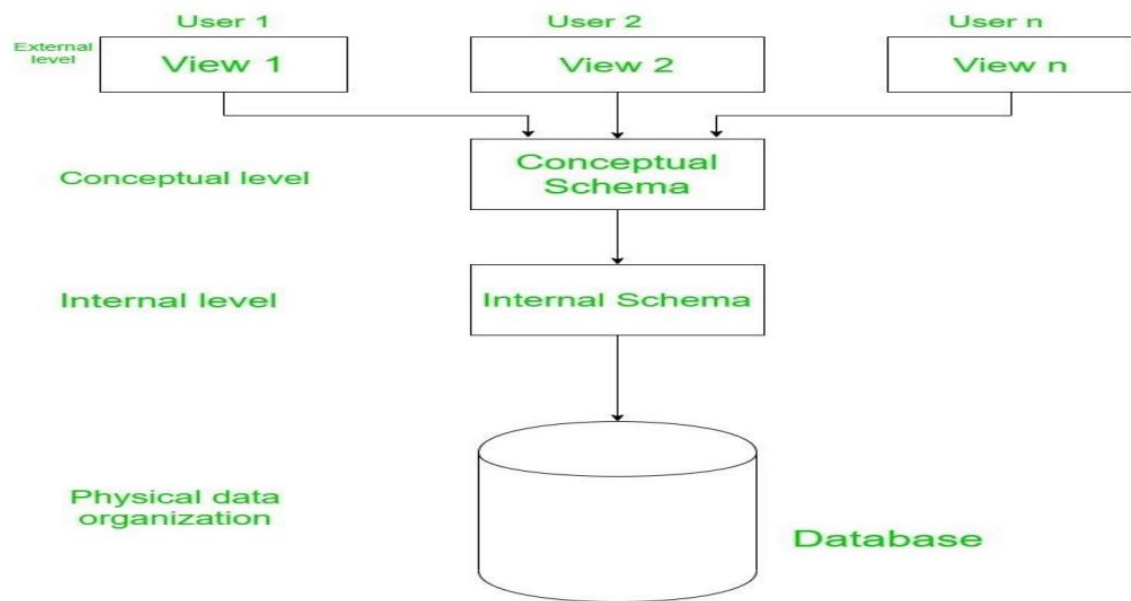
Faculty			
<u>Emp_Name</u>	Address	<u>Mobile_No</u>	Subject
Prof. Ajay Shah	Rajkot	9876543210	PPS

Metadata of above table is:

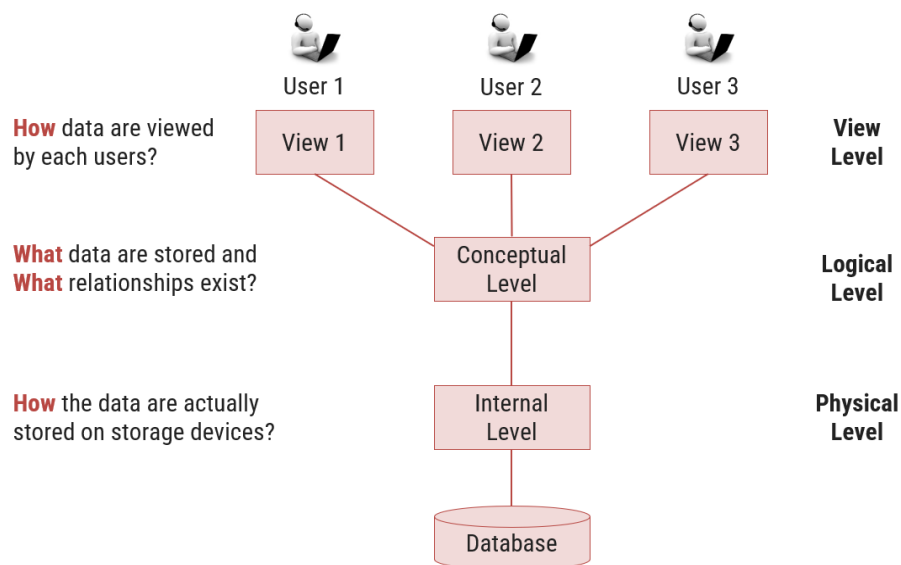
- Table name such as Faculty
- Column name such as Emp_Name, Address, Mobile_No, Subject
- Datatype such as Varchar, Decimal
- Access privileges such as Read, Write (Update)

3 Levels ANSI SPARC Database System

- ▶ **Internal level (Physical level) : Lowest level of the data abstraction**
 - ↳ It describes how a data is stored on the storage device.
 - ↳ Deals with physical storage of data.
 - Structure of records on disk - files, pages, blocks and indexes and ordering of records
 - ↳ Internal view is described by the internal schema.
- ▶ **Conceptual level (Logical level)**
 - ↳ What data are stored and what relationships exist among those data?
 - ↳ It hides low level complexities of physical storage.
 - ↳ For Example, STUDENT database may contain STUDENT and COURSE tables which will be visible to users but users are unaware about their storage.
 - ↳ Database administrator works at this level to determine what data to keep in the database.
- ▶ **External level (View level) : highest level of data abstraction**
 - ↳ It describes only part of the entire database that an end user concern or how data are viewed by each user.
 - ↳ Different user needs different views of the database, so there can be many views in a view level abstraction of the database. Used by end users and application programmers.
 - ↳ End users need to access only part of the database rather than the entire database.
 - ↳ External view is describes by external schema.



The ANSI-SPARC three-level architecture



DATABASE ARCHITECTURE

- **DBMS Architecture** refers to the way a Database Management System is organized and how users interact with the database.
- **DBMS Architecture** is the structural design that defines how users, applications, and the database interact. The common architectures are one-tier, two-tier, and three-tier, while the ANSI/SPARC architecture organizes data into external, conceptual, and internal levels to provide abstraction and data independence.

1. One-Tier Architecture

- ▶ The user, application, and database are all on the same system.
- ▶ Mainly used for development and testing.
- ▶ Diagram: User + Application + Database (Same Machine)
- ▶ Example: A developer using a local database on their own computer.

Advantages:

- ▶ Simple to set up.
- ▶ Fast because everything is on one machine.

Disadvantages:

- ▶ Not suitable for multiple users.
- ▶ Limited scalability and security.

2. Two-Tier Architecture (Client–Server)

- ▶ The **client application** communicates directly with the **database server**.
- ▶ Commonly used in small business applications.
- ▶ Diagram: | Client | -----> | Database Server |
 | (User App) | --> | (DBMS) |
- ▶ **Example:** A desktop application connected to a MySQL or Oracle database.

Advantages:

- ▶ Easy to implement.
- ▶ Better performance for small systems.

Disadvantages:

- ▶ Clients communicate directly with the database, which can raise security concerns.
- ▶ Less suitable for large-scale applications.

3. Three-Tier Architecture

The system is divided into **three layers**:

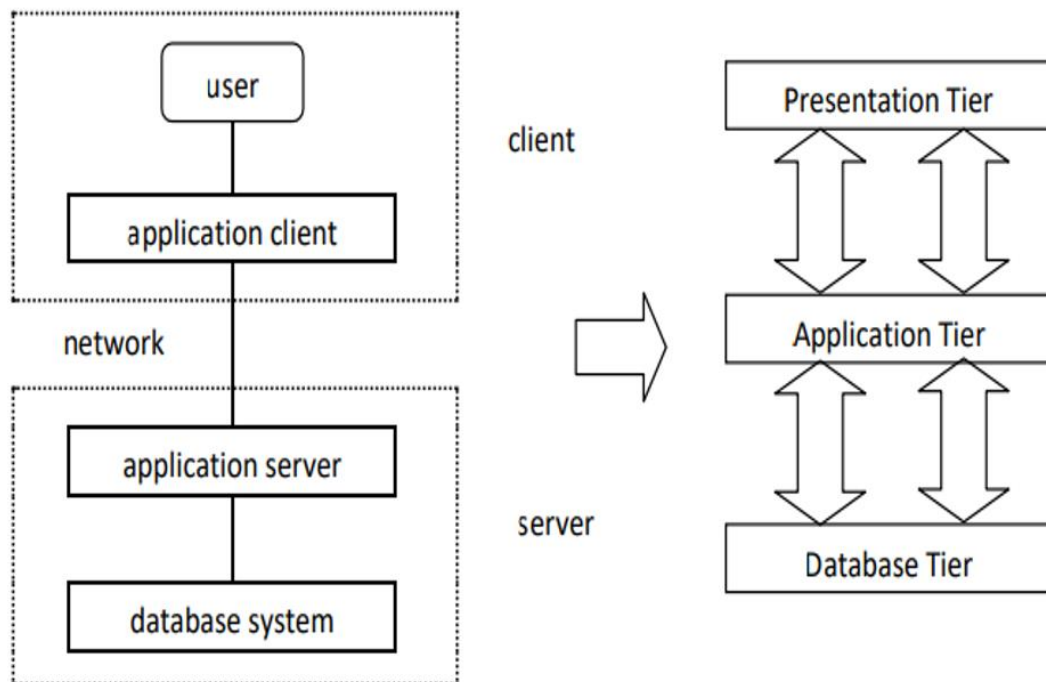
- ↪ **Presentation Layer (Client/User Interface)**
- ↪ **Application Layer (Business Logic)**
- ↪ **Database Layer (DBMS and Database)**
- ↪ **Example:** An online banking or e-commerce website where the browser talks to an application server, which then accesses the database.

Advantages:

- ▶ Improved security because users do not access the database directly.
- ▶ Easier to maintain and scale.
- ▶ Supports many users efficiently.

Disadvantages:

- ▶ More complex to design.
- ▶ Requires additional servers and management.



Database (Data) Tier

- At this tier, only database resides.
- Database along with its query processing languages sits in layer-3 of 3-tier architecture.
- It also contains all relations and their constraints.

Application (Middle) Tier

- At this tier, the application server and program, which access database, resides.
- For a user this application tier works as abstracted view of database.
- Users are unaware of any existence of database beyond application.
- For database-tier, application tier is the user of it.
- Database tier is not aware of any other user beyond application tier.
- This tier works as mediator between the two.

User (Presentation) Tier

- An end user sits on this tier.
- From a user's aspect, this tier is everything.
- He/she doesn't know about any existence or form of database beyond this layer.
- At this layer multiple views of database can be provided by the application.
- All views which are generated by an application, resides in application tier.

Summary of Database architecture

Architecture	Layers	Best For	Main Benefit
One-Tier	1	Development and testing	Simple and fast
Two-Tier	2	Small client-server applications	Direct communication with the database
Three-Tier	3	Web and enterprise applications	Better security, scalability, and maintenance

Data Independence

- ▶ Data Independence : Data independency is the ability to modify a schema definition in one level without affecting a schema definition in the next higher level.
- ▶ Types of data independence :
 - (1) Physical data independence
 - (2) Logical data independence
- ▶ Physical Data Independence
 - Physical Data Independence is the ability to modify the physical schema without requiring any change in logical (conceptual) schema and application programs.
 - Physical data independence allows changing in physical storage devices or organization of file without change in the conceptual view or external view.
 - Modifications at the internal levels are occasionally necessary to improve performance.
 - Possible modifications at internal levels are changes in file structures, compression techniques, hashing algorithms, storage devices, etc.
 - Physical data independence separates conceptual level from the internal level.
 - It is easy to achieve physical data independence.
- ▶ Logical Data Independence
 - Logical data independence is the ability to modify the conceptual schema without requiring any change in application programs.
 - Conceptual schema can be changed without affecting the existing external schema.
 - Modification at the logical levels is necessary whenever the logical structure of the database is changed.
 - Application programs are heavily dependent on logical structures of the data they access. So, any change in logical structure also requires programs to change.

- ↳ Logical data independence separates external level from the conceptual view.
- ↳ It is difficult to achieve logical data independence.

Client-Server Architecture -

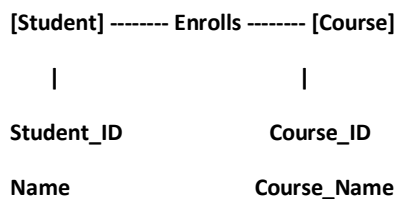
- System Design Client-server architecture is a widely used system design where multiple clients request services or resources from a central server. The server manages processing, data storage, and resources, while clients handle user interaction.
- Clients communicate with servers using protocols like HTTP/HTTPS or SQL to send requests and receive responses.
- Servers handle centralized data management, processing, and storage, ensuring efficient system operation.
- The architecture supports scalability (vertical and horizontal) and is widely used in web apps, databases, and email systems.

Entity-relationship Model

Entity-Relationship (ER) Model

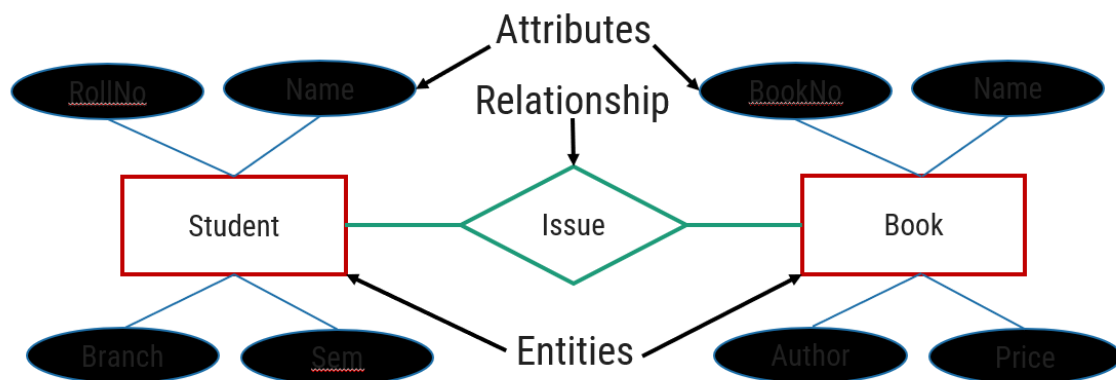
- ▶ Used mainly for database design.
- ▶ Represents data using entities, attributes, and relationships.

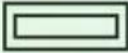
Example:



Advantages:

- ▶ Easy to visualize and understand.
- ▶ Helps in designing relational databases.
- ▶ In this database model, we represent the real-world problem in the pictorial form to make it easy for the stakeholders to understand.
- ▶ It is also very easy for the developers to understand the system by just looking at the ER diagram.



Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

key concepts of DBMS

Database Management System (DBMS) is software used to create, store, organize, and manage data efficiently.

1. Data

- ▶ Raw facts and figures stored in the database.
- ▶ Example: 101, Rahul, 85.

2. Database

- ▶ A collection of related data organized in a structured way.
- ▶ Example: A college database containing student, faculty, and course information.

3. Record (Row/Tuple)

- ▶ A single row in a table representing one complete entry.
- ▶ Example: (101, Rahul, Ahmedabad).

4. Table

- ▶ Data is stored in the form of rows and columns.

Example:

5. Field (Column/Attribute)

- ▶ A column that stores a specific type of information.
- ▶ Example: Student_ID, Name, City.

6. Primary Key

- ▶ A field that uniquely identifies each record in a table.
- ▶ It cannot contain duplicate or NULL values.

- ▶ Example: Student_ID.

7. Foreign Key

- ▶ A field that links one table to another by referencing the primary key of another table.
- ▶ Example: Course_ID in an enrollment table referring to the course table.

8. Entity

- ▶ A real-world object or thing about which data is stored.
- ▶ Examples: Student, Employee, Product.

9. Attribute

- ▶ A property or characteristic of an entity.
- ▶ Example: A Student entity may have attributes such as Student_ID, Name, and Age.

10. Relationship

- ▶ Describes how entities are connected.
- ▶ Example: A student enrolls in a course.

11. Schema

- ▶ The overall structure or design of the database, including tables, columns, and relationships

12. Query

- ▶ A request to retrieve or manipulate data from the database.
- ▶ Example: `SELECT * FROM STUDENT;`

13. DBMS

- ▶ Software that manages databases and provides functions such as storing, retrieving, updating, and securing data.
- ▶ Examples: MySQL, Oracle Database, PostgreSQL, Microsoft SQL Server.

14. Normalization

- ▶ The process of organizing data to reduce redundancy and improve consistency.

15. Constraints

- ▶ Rules applied to ensure data accuracy and integrity.
- ▶ Examples: PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK.
- ▶ Key concepts of DBMS are the fundamental elements used to store, organize, manage, and retrieve data effectively. These include data, database, tables, records, fields, keys, entities, attributes, relationships, schema, queries, normalization, and constraints.

Concept	Simple Meaning	Example
Data	Raw facts	101, Rahul
Database	Organized collection of data	College database
Table	Data in rows and columns	STUDENT table
Record	One row in a table	(101, Rahul, Ahmedabad)
Field	One column in a table	Name
Primary Key	Unique identifier	Student_ID
Foreign Key	Connects tables	Course_ID
Entity	Real-world object	Student
Attribute	Property of an entity	Age
Relationship	Connection between entities	Student enrolls in Course
Schema	Database design	Table definitions
Query	Request for data	SELECT * FROM STUDENT;
Normalization	Reduces duplicate data	Splitting related data into separate tables
Constraints	Rules for valid data	NOT NULL, UNIQUE

